



Database System

GROUP ASSIGNMENT

Hotel Reservation Management System

Academic Honesty Policy Statement

I hereby attest those contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Submitted By Group 3

Name : Outhorninvithyea ID : 601007764	Name : Kong Dymond ID : 601007754	Name : Chantha Sinat ID : 601007149
Name : Chhoeng Dyne ID : 601007780	Name : ID :	Name: ID :

Under the supervision of Lectures Leng Rathana

Table of Content

Chapter 1: Introduction.....	3
1.1 Background of the System.....	3
1.2 Objectives of the Database System.....	3
1.3 Scope of the System.....	4
Chapter 2: Database Design.....	5
2.1 ERD Diagram.....	5
2.2 Normalization Process.....	5
1. Unnormalized Form (UNF).....	5
2. First Normal Form (1NF).....	6
_ Rule of 1NF.....	6
_ Instead of:.....	6
_ We create separate rows:.....	6
_ Example (1NF):.....	7
_ Primary Key could be:.....	7
3. Second Normal Form (2NF).....	7
_ Rule of 2NF.....	7
_ If the primary key is:.....	7
4. Third Normal Form (3NF).....	10
_ Rule of 3NF.....	10
Example:.....	10
➤ Final Tables in System (3NF).....	10
Chapter 2.3: Data Dictionary.....	11
1. Table Name: Guests.....	12
2. Table Name: RoomTypes.....	12
3. Table Name: Rooms.....	13
4. Table Name: Reservations.....	14
Chapter 3: Implementation and Loading.....	15
3.1 CREATE TABLE statements.....	15
3.2 INSERT statements with sample data.....	19
Chapter 4: Testing and Evaluation.....	21
Chapter 5: Conclusion & Future Enhancements.....	26
5.1 Summary of the database system.....	26
5.2 Suggested future improvements or extensions.....	27

Chapter 1: Introduction

1.1 Background of the System

The hotel industry has grown a lot in recent years. Many hotels need a better and faster way to manage reservations and daily operations. Traditional manual booking systems are slow, easy to make mistakes, and cannot show real-time room availability.

A Hotel Reservation Management System is a database system that helps manage the whole booking process, from making a reservation to guest checkout. The system manages important hotel operations such as room information, customer details, staff assignments, payments, and extra service requests. This system helps solve common problems such as double booking, lost reservation records, slow billing, and difficulty tracking customer history. By using a proper database system, hotels can improve efficiency, increase customer satisfaction, and manage resources better.

1.2 Objectives of the Database System

The main objectives of this system are:

- **Centralized Data Management**
Store all hotel information in one organized database.
- **Efficient Reservation Processing**
Allow quick and accurate room booking, editing, and cancellation.
- **Customer Management**
Keep customer history, preferences, and loyalty information.
- **Revenue Monitoring**
Track room occupancy and pricing.
- **Staff Management**
Assign staff members to handle reservations and guest services.
- **Payment Tracking**
Record all payments correctly and clearly.
- **Service Management**
Manage additional services requested by guests.
- **Reporting**
Generate reports to help management make decisions.

1.3 Scope of the System

Included Features

The system includes:

- Customer registration and profile management
- Room information and room types
- Reservation booking, update, and cancellation
- Staff assignment
- Payment recording
- Additional service requests
- Room availability tracking
- Multi-table queries for reports

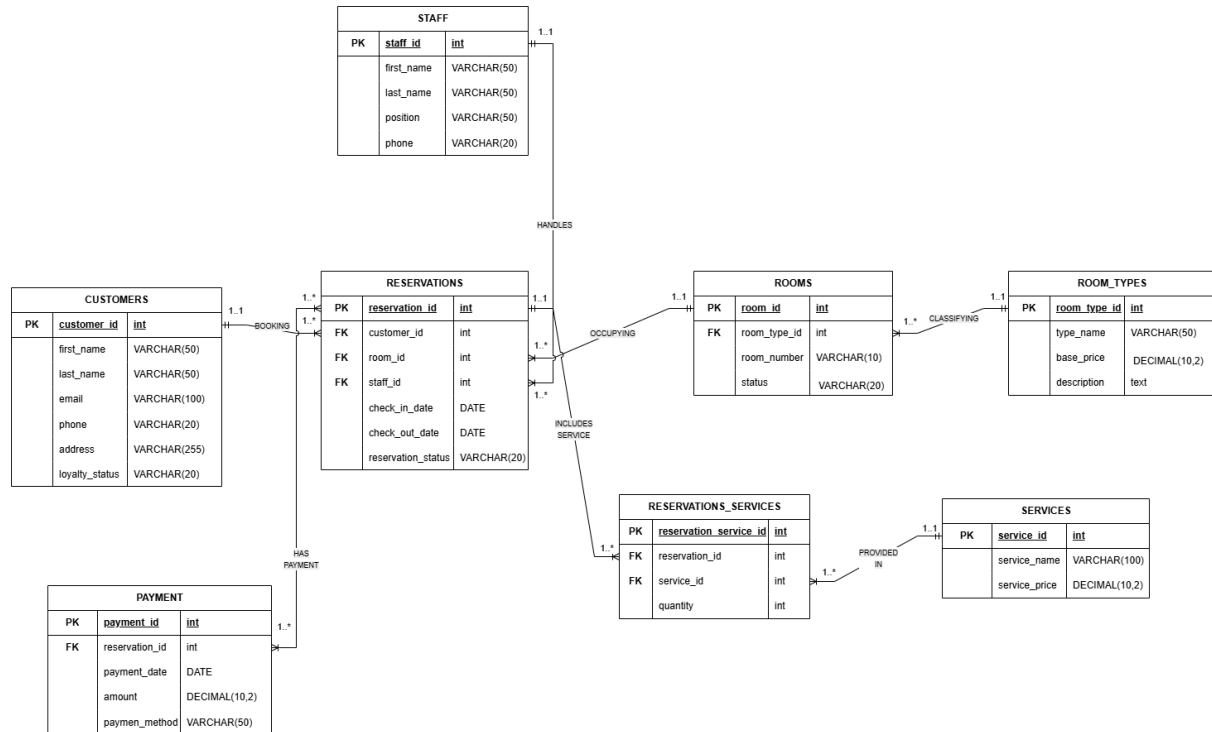
Excluded Features

The system does not include:

- Online booking website
- Email notification system
- Credit card payment gateway
- Housekeeping schedule
- Maintenance tracking
- Hotel supply inventory management

Chapter 2: Database Design

2.1 ERD Diagram



2.2 Normalization Process

1. Unnormalized Form (UNF)

Imagine if we stored everything in one big table like this:

reservation_id	guest_name	phone	room_number	room_type	room_price
1	James	012-345-678	101	Single	60

check-in	check-out	service_name	service_price	staff_name
2026-02-23	2026-02-26	Breakfast, Massage	25-80	Sarah

+ Problems in UNF:

1. Data redundancy
 - Guest name repeated for every reservation
 - Room type repeated for every room
 - Service price repeated many times
2. Update anomaly
 - If room price changes → must update many rows
3. Insert anomaly
 - Cannot insert new service unless there is a reservation
4. Delete anomaly
 - If we delete last reservation → guest information is lost
5. Multiple values in one column
 - "Breakfast, Massage" (not atomic)

This structure is NOT organized.

2. First Normal Form (1NF)

_ Rule of 1NF

- Each column must contain atomic (single) values
- No repeating groups
- Must have a primary key

_ Instead of:

service_name = Breakfast, Massage

_ We create separate rows:

reservation_id	service_name
1	Breakfast
1	Massage

_ Example (1NF):

reservation_id	guest_name	room_number	service_name
1	James	101	Breakfast
1	James	101	Massage

_ Primary Key could be:

(reservation_id, service_name)

_ Problem Still Exists

- Guest name depends only on reservation_id
- Room type depends only on room_number
- Service price depends only on service_name

This creates **partial dependency**.

3. Second Normal Form (2NF)

_ Rule of 2NF

- Must already be in 1NF
- No partial dependency
- All non-key attributes must depend on the FULL primary key

_ If the primary key is:

(reservation_id, service_name)

- guest_name depends only on reservation_id
- service_price depends only on service_name

Now we separate data into proper tables.

```
CUSTOMERS (  
    customer_id PK,  
    first_name,  
    last_name,  
    email,  
    phone,  
    address,  
    loyalty_status  
)
```

```
STAFF (  
    staff_id PK,  
    first_name,  
    last_name,  
    position,  
    phone  
)
```

```
ROOM_TYPES (  
    room_type_id PK,  
    type_name,  
    base_price,  
    description  
)
```

```
ROOMS (  
    room_id PK,  
    room_type_id FK,  
    room_number,  
    status  
)
```

```
RESERVATIONS (  
    reservation_id PK,  
    customer_id FK,  
    room_id FK,  
    staff_id FK,  
    check_in_date,  
    check_out_date,  
    reservation_status  
)
```

```
SERVICES (  
    service_id PK,  
    service_name,  
    service_price  
)
```

```
RESERVATIONS_SERVICES (  
    reservation_service_id PK,  
    reservation_id FK,  
    service_id FK,  
    quantity  
)
```

```
PAYMENT (  
    payment_id PK,  
    reservation_id FK,  
    payment_date,  
    amount,  
    payment_method
```

)

- Customers info depends only on customer_id
- Room type depends only on room_type_id
- Service price depends only on service_id

4. Third Normal Form (3NF)

_ Rule of 3NF

- Must be in 2NF
- No transitive dependency
- Non-key attributes must NOT depend on other non-key attributes

Example:

```
Reservation (  
    reservation_id PK,  
    customer_id FK,  
    room_id FK,  
)
```

Here:

- reservation_id → customer_id
- customer_id → room_id

So:

reservation_id → room_id (indirectly)

➤ Final Tables in System (3NF)

Final database is already in 3NF:

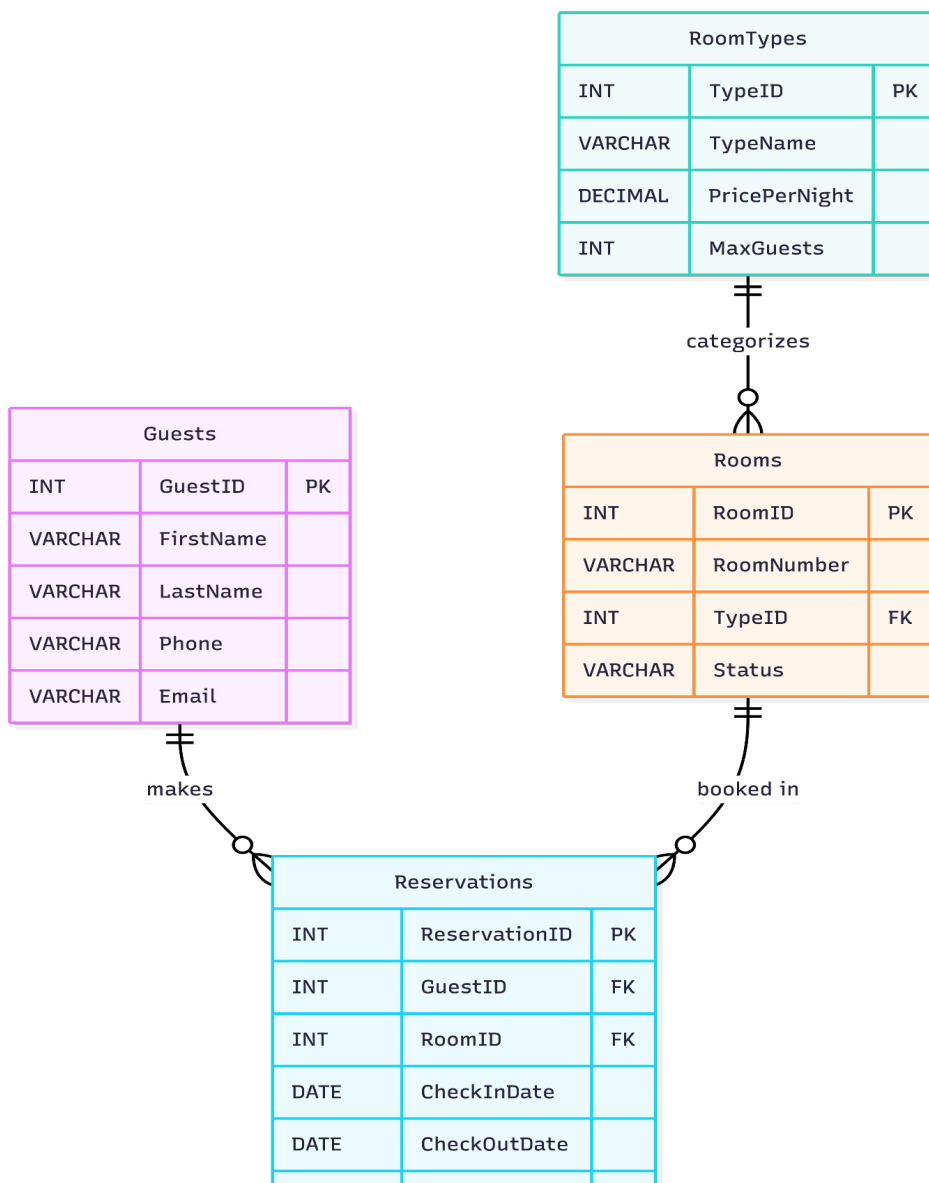
1. Room_Type
2. Room

3. Customers
4. Staff
5. Service
6. Reservation
7. Payment
8. Reservation_Service

Each table:

- Has a primary key
- Has atomic columns
- No repeating groups
- No partial dependency
- No transitive dependency

Chapter 2.3: Data Dictionary



1. Table Name: Guests

Attribute Name	Data Type	Key	Description
GuestID	INT	PK	Unique auto-incremented ID for each guest.
FirstName	VARCHAR(50)		The guest's given name.
LastName	VARCHAR(50)		The guest's family name.
Phone	VARCHAR(20)		The guest's contact phone number.
Email	VARCHAR(100)		The guest's email address.

2. Table Name: RoomTypes

Attribute Name	Data Type	Key	Description
TypeID	INT	PK	Unique auto-incremented ID for each room category.

TypeName	VARCHAR(50)		The name of the room type (e.g., Standard Single, Executive Suite).
PricePerNight	DECIMAL(10,2)		The cost to rent the room for one night.
MaxGuests	INT		The maximum number of people allowed in this room type.

3. Table Name: Rooms

Attribute Name	Data Type	Key	Description
RoomID	INT	PK	Unique auto-incremented internal ID for the physical room.
RoomNumber	VARCHAR(10)		The actual number on the hotel door (e.g., "101").
TypeID	INT	FK	Links to the RoomTypes table to determine price and capacity.

Status	VARCHAR(20)		Current state of the room (e.g., Available, Occupied, Maintenance).
---------------	--------------------	--	--

4. Table Name: Reservations

Attribute Name	Data Type	Key	Description
ReservationID	INT	PK	Unique auto-incremented booking reference number.
GuestID	INT	FK	Links to the Guests table to identify who made the booking.
RoomID	INT	FK	Links to the Rooms table to identify which room is booked.
CheckInDate	DATE		The date the guest arrives.
CheckOutDate	DATE		The date the guest is scheduled to leave.
TotalAmount	DECIMAL(10,2)		The total calculated cost of the stay.

Chapter 3: Implementation and Loading

3.1 CREATE TABLE statements

```
-- =====
-- CREATE DATABASE
-- =====

• CREATE DATABASE IF NOT EXISTS hotel_reservation_db;
• USE hotel_reservation_db;

-- =====
-- CREATE TABLES
-- =====

-- Room_Type Table
-- =====

• CREATE TABLE IF NOT EXISTS Room_Type (
    room_type_id INT AUTO_INCREMENT PRIMARY KEY,
    type_name    VARCHAR(50)    NOT NULL,
    description  TEXT,
    base_price   DECIMAL(10,2)  NOT NULL,
    max_occupancy INT           NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS Room (
    room_id      INT AUTO_INCREMENT PRIMARY KEY,
    room_number  VARCHAR(10)  NOT NULL UNIQUE,
    room_type_id INT          NOT NULL,
    floor_number INT          NOT NULL,
    status       VARCHAR(20)  NOT NULL,
    CONSTRAINT chk_room_status CHECK (status IN ('Available', 'Occupied', 'Maintenance')),
    FOREIGN KEY (room_type_id) REFERENCES Room_Type(room_type_id)
);
```

```
-- Guest Table
-- =====
```

```
CREATE TABLE IF NOT EXISTS Guest (
    guest_id     INT AUTO_INCREMENT PRIMARY KEY,
    first_name   VARCHAR(50)  NOT NULL,
    last_name    VARCHAR(50)  NOT NULL,
    email        VARCHAR(100),
    phone        VARCHAR(20),
    nationality   VARCHAR(50),
    id_number    VARCHAR(50)
);
```

```
-- Staff Table
```

```
CREATE TABLE IF NOT EXISTS Staff (  
    staff_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100),  
    phone VARCHAR(20),  
    position VARCHAR(100),  
    department VARCHAR(100),  
    hire_date DATE  
);
```

```
-- Service Table
```

```
CREATE TABLE IF NOT EXISTS Service (  
    service_id INT AUTO_INCREMENT PRIMARY KEY,  
    service_name VARCHAR(100) NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL,  
    category VARCHAR(50)  
);
```

```
-- Reservation Table
```

```
• ○ CREATE TABLE IF NOT EXISTS Reservation (  
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
    guest_id INT NOT NULL,  
    room_id INT NOT NULL,  
    staff_id INT NOT NULL,  
    check_in_date DATE NOT NULL,  
    check_out_date DATE NOT NULL,  
    num_guests INT NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    special_requests TEXT,  
    CONSTRAINT chk_reservation_status  
        CHECK (status IN ('Confirmed', 'Checked-In', 'Checked-Out', 'Cancelled')),  
    CONSTRAINT chk_checkout_after_checkin  
        CHECK (check_out_date > check_in_date),  
    FOREIGN KEY (guest_id) REFERENCES Guest(guest_id),  
    FOREIGN KEY (room_id) REFERENCES Room(room_id),  
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)  
);
```

```
-----  
-- Guest Table  
-----
```

```
• ○ CREATE TABLE IF NOT EXISTS Guest (  
    guest_id    INT AUTO_INCREMENT PRIMARY KEY,  
    first_name  VARCHAR(50)  NOT NULL,  
    last_name   VARCHAR(50)  NOT NULL,  
    email       VARCHAR(100),  
    phone       VARCHAR(20),  
    nationality  VARCHAR(50),  
    id_number   VARCHAR(50)  
);
```

```
-----  
-- Staff Table  
-----
```

```
• ○ CREATE TABLE IF NOT EXISTS Staff (  
    staff_id    INT AUTO_INCREMENT PRIMARY KEY,  
    first_name  VARCHAR(50)  NOT NULL,  
    last_name   VARCHAR(50)  NOT NULL,  
    email       VARCHAR(100),  
    phone       VARCHAR(20),  
    position    VARCHAR(100),  
    department  VARCHAR(100),  
    hire_date   DATE  
);
```

```
-----  
-- Service Table  
-----
```

```
• ○ CREATE TABLE IF NOT EXISTS Service (  
    service_id  INT AUTO_INCREMENT PRIMARY KEY,  
    service_name VARCHAR(100)  NOT NULL,  
    description  TEXT,  
    price       DECIMAL(10,2) NOT NULL,  
    category    VARCHAR(50)  
);
```

-- Reservation Table

```
CREATE TABLE IF NOT EXISTS Reservation (  
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
    guest_id      INT NOT NULL,  
    room_id       INT NOT NULL,  
    staff_id      INT NOT NULL,  
    check_in_date DATE NOT NULL,  
    check_out_date DATE NOT NULL,  
    num_guests    INT NOT NULL,  
    total_amount  DECIMAL(10,2) NOT NULL,  
    status        VARCHAR(20) NOT NULL,  
    special_requests TEXT,  
    CONSTRAINT chk_reservation_status  
        CHECK (status IN ('Confirmed', 'Checked-In', 'Checked-Out', 'Cancelled')),  
    CONSTRAINT chk_checkout_after_checkin  
        CHECK (check_out_date > check_in_date),  
    FOREIGN KEY (guest_id) REFERENCES Guest(guest_id),  
    FOREIGN KEY (room_id) REFERENCES Room(room_id),  
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)  
);
```

-- Payment Table

```
CREATE TABLE IF NOT EXISTS Payment (  
    payment_id      INT AUTO_INCREMENT PRIMARY KEY,  
    reservation_id  INT NOT NULL,  
    amount          DECIMAL(10,2) NOT NULL,  
    payment_date    DATETIME NOT NULL DEFAULT NOW(),  
    payment_method  VARCHAR(50) NOT NULL,  
    payment_status  VARCHAR(20) NOT NULL,  
    transaction_ref VARCHAR(50),  
    FOREIGN KEY (reservation_id) REFERENCES Reservation(reservation_id)  
);
```

-- Reservation_Service Table

```
CREATE TABLE IF NOT EXISTS Reservation_Service (  
    reservation_service_id INT AUTO_INCREMENT PRIMARY KEY,  
    reservation_id         INT NOT NULL,  
    service_id             INT NOT NULL,  
    quantity               INT NOT NULL DEFAULT 1,  
    service_date           DATE NOT NULL,  
    FOREIGN KEY (reservation_id) REFERENCES Reservation(reservation_id),  
    FOREIGN KEY (service_id) REFERENCES Service(service_id)  
);
```

3.2 INSERT statements with sample data

```
-- =====
-- SECTION 1: INSERT SAMPLE DATA
-- =====

-- -----
-- Clear existing data (in order to respect foreign keys)
-- -----

SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE Reservation_Service;
TRUNCATE TABLE Payment;
TRUNCATE TABLE Reservation;
TRUNCATE TABLE Room;
TRUNCATE TABLE Room_Type;
TRUNCATE TABLE Guest;
TRUNCATE TABLE Staff;
TRUNCATE TABLE Service;
SET FOREIGN_KEY_CHECKS = 1;

-- -----
-- 5 Room Types
-- -----

INSERT INTO Room_Type (type_name, description, base_price, max_occupancy) VALUES
('Single',      'Cozy room with one single bed, ideal for solo travelers',      60.00, 1),
('Double',      'Comfortable room with one double bed for couples or solo guests', 95.00, 2),
('Suite',       'Spacious suite with separate living area and premium amenities', 220.00, 2),
('Family Room', 'Large room with two beds, perfect for families with children',   150.00, 4),
('Deluxe Suite', 'Luxury suite with jacuzzi, sea view, and butler service',       380.00, 2);

-- 20 Rooms
-- -----

INSERT INTO Room (room_number, room_type_id, floor_number, status) VALUES
-- Floor 1: Single rooms (type 1)
('101', 1, 1, 'Available'),
('102', 1, 1, 'Occupied'),
('103', 1, 1, 'Available'),
('104', 1, 1, 'Maintenance'),
-- Floor 2: Double rooms (type 2)
('201', 2, 2, 'Available'),
('202', 2, 2, 'Occupied'),
('203', 2, 2, 'Available'),
('204', 2, 2, 'Available'),
-- Floor 2: Family Rooms (type 4)
('205', 4, 2, 'Available'),
('206', 4, 2, 'Occupied'),
-- Floor 3: Suite rooms (type 3)
('301', 3, 3, 'Available'),
('302', 3, 3, 'Occupied'),
('303', 3, 3, 'Available'),
('304', 3, 3, 'Available'),
-- Floor 3: Family Rooms (type 4)
('305', 4, 3, 'Available'),
('306', 4, 3, 'Available'),
-- Floor 4: Deluxe Suites (type 5)
('401', 5, 4, 'Available'),
('402', 5, 4, 'Occupied'),
('403', 5, 4, 'Available'),
('404', 5, 4, 'Maintenance');
```

```

-----
-- 10 Guests
-----
INSERT INTO Guest (first_name, last_name, email, phone, nationality, id_number) VALUES
('James', 'Anderson', 'james.anderson@gmail.com', '012-345-6789', 'American', 'US123456'),
('Sophie', 'Laurent', 'sophie.laurent@gmail.com', '012-456-7890', 'French', 'FR789012'),
('Kenji', 'Tanaka', 'kenji.tanaka@yahoo.com', '011-567-8901', 'Japanese', 'JP345678'),
('Maria', 'Garcia', 'maria.garcia@gmail.com', '015-678-9012', 'Spanish', 'ES901234'),
('David', 'Nguyen', 'david.nguyen@yahoo.com', '012-789-0123', 'Vietnamese', 'VN567890'),
('Emma', 'Williams', 'emma.williams@email.com', '017-890-1234', 'British', 'GB123789'),
('Ali', 'Hassan', 'ali.hassan@gmail.com', '012-901-2345', 'Emirati', 'AE456012'),
('Priya', 'Sharma', 'priya.sharma@email.com', '011-012-3456', 'Indian', 'IN789345'),
('Carlos', 'Mendez', 'carlos.mendez@yahoo.com', '015-123-4567', 'Mexican', 'MX012678'),
('Lin', 'Chen', 'lin.chen@gmail.com', '012-234-5678', 'Chinese', 'CN345901');

-----

-- 8 Staff Members
-----
INSERT INTO Staff (first_name, last_name, email, phone, position, department, hire_date) VALUES
('Sarah', 'Johnson', 'sarah.johnson@hotel.com', '010-111-2222', 'Front Desk Agent', 'Reception', '2023-01-15'),
('Michael', 'Brown', 'michael.brown@hotel.com', '010-222-3333', 'Front Desk Manager', 'Reception', '2022-06-01'),
('Emily', 'Davis', 'emily.davis@hotel.com', '010-333-4444', 'Concierge', 'Guest Services', '2023-03-10'),
('Robert', 'Wilson', 'robert.wilson@hotel.com', '010-444-5555', 'Reservation Manager', 'Reception', '2022-11-20'),
('Lisa', 'Martinez', 'lisa.martinez@hotel.com', '010-555-6666', 'Front Desk Agent', 'Reception', '2023-05-05'),
('James', 'Taylor', 'james.taylor@hotel.com', '010-666-7777', 'Guest Relations', 'Guest Services', '2023-02-14'),
('Anna', 'Anderson', 'anna.anderson@hotel.com', '010-777-8888', 'Front Desk Agent', 'Reception', '2023-07-22'),
('David', 'Thomas', 'david.thomas@hotel.com', '010-888-9999', 'Concierge', 'Guest Services', '2023-04-18');

-----

-- 9 Services
-----
INSERT INTO Service (service_name, description, price, category) VALUES
('Room Service Breakfast', 'Full breakfast delivered to your room', 25.00, 'Dining'),
('Room Service Dinner', 'Gourmet dinner served in your room', 45.00, 'Dining'),
('Mini Bar Restocking', 'Restock your mini bar with selected items', 30.00, 'Room Service'),
('Swedish Massage', 'Relaxing 60-minute Swedish massage', 80.00, 'Spa'),
('Facial Treatment', 'Rejuvenating facial treatment', 75.00, 'Spa'),
('Laundry Service', 'Professional laundry and dry cleaning', 35.00, 'Housekeeping'),
('Airport Transfer', 'Private car transfer to/from airport', 50.00, 'Transportation'),
('City Tour', 'Guided city tour with professional guide', 120.00, 'Activities'),
('Extra Bed', 'Additional bed setup in your room', 40.00, 'Room Service');

-----

-- 15 Reservations
-----
INSERT INTO Reservation (
    guest_id, room_id, staff_id, check_in_date, check_out_date, num_guests, total_amount, status, special_requests
) VALUES
-- January 2025
(1, 2, 2, '2025-01-05', '2025-01-07', 1, 120.00, 'Checked-Out', NULL),
(2, 6, 1, '2025-01-10', '2025-01-13', 2, 285.00, 'Checked-Out', 'Non-smoking room'),
(3, 12, 2, '2025-01-15', '2025-01-17', 1, 440.00, 'Checked-Out', 'Late checkout please'),
-- February 2025
(4, 10, 5, '2025-02-01', '2025-02-04', 3, 450.00, 'Checked-Out', 'Airport pickup needed'),
(5, 18, 1, '2025-02-05', '2025-02-07', 2, 760.00, 'Checked-Out', NULL),
(6, 1, 2, '2025-02-10', '2025-02-12', 1, 120.00, 'Checked-Out', 'Early check-in request'),
(7, 5, 5, '2025-02-14', '2025-02-16', 2, 190.00, 'Checked-Out', 'Valentine decoration'),
(8, 15, 2, '2025-02-20', '2025-02-23', 4, 450.00, 'Checked-Out', 'Baby cot required'),
-- March 2025
(9, 13, 1, '2025-03-01', '2025-03-03', 2, 440.00, 'Checked-Out', NULL),
(10, 19, 5, '2025-03-05', '2025-03-08', 2, 1140.00, 'Checked-Out', 'Champagne on arrival'),
(1, 3, 2, '2025-03-10', '2025-03-12', 1, 120.00, 'Checked-Out', NULL),
(2, 7, 1, '2025-03-18', '2025-03-21', 2, 285.00, 'Checked-Out', 'High floor preferred'),
-- April 2025 (upcoming)
(3, 11, 2, '2025-04-01', '2025-04-04', 1, 660.00, 'Confirmed', 'Quiet room please'),
(4, 16, 5, '2025-04-05', '2025-04-08', 4, 450.00, 'Confirmed', NULL),
(5, 17, 1, '2025-04-10', '2025-04-12', 2, 760.00, 'Confirmed', 'Sea view room requested');

```

```
-- Payments (one per reservation)
```

```
INSERT INTO Payment (reservation_id, amount, payment_date, payment_method, payment_status, transaction_ref) VALUES
(1, 120.00, '2025-01-07 10:00:00', 'Cash', 'Completed', 'TXN-CA-001'),
(2, 285.00, '2025-01-13 11:30:00', 'Credit Card', 'Completed', 'TXN-CC-002'),
(3, 440.00, '2025-01-17 09:00:00', 'Online', 'Completed', 'TXN-ON-003'),
(4, 450.00, '2025-02-04 14:00:00', 'Bank Transfer', 'Completed', 'TXN-BT-004'),
(5, 760.00, '2025-02-07 12:00:00', 'Credit Card', 'Completed', 'TXN-CC-005'),
(6, 120.00, '2025-02-12 10:30:00', 'Cash', 'Completed', 'TXN-CA-006'),
(7, 190.00, '2025-02-16 15:00:00', 'Debit Card', 'Completed', 'TXN-DC-007'),
(8, 450.00, '2025-02-23 09:30:00', 'Online', 'Completed', 'TXN-ON-008'),
(9, 440.00, '2025-03-03 11:00:00', 'Credit Card', 'Completed', 'TXN-CC-009'),
(10, 1140.00, '2025-03-08 13:00:00', 'Bank Transfer', 'Completed', 'TXN-BT-010'),
(11, 120.00, '2025-03-12 10:00:00', 'Cash', 'Completed', 'TXN-CA-011'),
(12, 285.00, '2025-03-21 14:30:00', 'Credit Card', 'Completed', 'TXN-CC-012'),
(13, 660.00, '2025-04-01 08:00:00', 'Credit Card', 'Pending', 'TXN-CC-013'),
(14, 450.00, '2025-04-05 09:00:00', 'Online', 'Pending', 'TXN-ON-014'),
(15, 760.00, '2025-04-10 08:30:00', 'Bank Transfer', 'Pending', 'TXN-BT-015');
```

```
-- Reservation Services
```

```
INSERT INTO Reservation_Service (reservation_id, service_id, quantity, service_date) VALUES
(2, 1, 2, '2025-01-11'),
(3, 4, 1, '2025-01-16'),
(4, 7, 1, '2025-02-01'),
(4, 9, 1, '2025-02-02'),
(5, 4, 1, '2025-02-06'),
(5, 2, 1, '2025-02-06'),
(7, 3, 1, '2025-02-15'),
(8, 9, 1, '2025-02-21'),
(9, 5, 1, '2025-03-02'),
(10, 8, 1, '2025-03-06'),
(10, 4, 2, '2025-03-07'),
(10, 2, 2, '2025-03-07'),
(11, 6, 1, '2025-03-11'),
(12, 1, 3, '2025-03-19');
```

Chapter 4: Testing and Evaluation

Query 1: [4-TABLE JOIN] List all guests who booked a Suite

- This query shows every guest who has reserved a Suite room.
- It helps the hotel identify premium guests for loyalty rewards and special offers.

```

SELECT
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.phone,
    g.nationality,
    rt.type_name AS room_type,
    r.check_in_date,
    r.check_out_date,
    r.status
FROM Reservation r
JOIN Guest g ON r.guest_id = g.guest_id
JOIN Room ro ON r.room_id = ro.room_id
JOIN Room_Type rt ON ro.room_type_id = rt.room_type_id
WHERE rt.type_name = 'Suite'
ORDER BY r.check_in_date;

```

Query 2: [AGGREGATE - SUM] Total revenue from reservations in February 2025

- This query calculates the total money the hotel earned during February 2025.
- It is useful for monthly financial reporting and tracking business performance.

```

SELECT
    MONTH(check_in_date) AS month_num,
    MONTHNAME(check_in_date) AS month_name,
    COUNT(reservation_id) AS total_reservations,
    SUM(total_amount) AS total_revenue
FROM Reservation
WHERE MONTH(check_in_date) = 2
    AND YEAR(check_in_date) = 2025
    AND status != 'Cancelled'
GROUP BY MONTH(check_in_date), MONTHNAME(check_in_date);

```

Query 3: [2-TABLE JOIN] Show all available rooms with their type and price

- This query lists every room that is currently free and ready to be booked.
- Front desk staff use this to quickly find and offer options to walk-in guests.

```

SELECT
    ro.room_number,
    rt.type_name      AS room_type,
    ro.floor_number,
    ro.status,
    rt.base_price     AS price_per_night,
    rt.max_occupancy
FROM Room            ro
JOIN Room_Type rt ON ro.room_type_id = rt.room_type_id
WHERE ro.status = 'Available'
ORDER BY rt.base_price ASC;

```

Query 4: [REGEXP] Find guests who use a Gmail email address

- This query finds all guests who signed up using a Gmail address.
- It helps the hotel reach out to Gmail users with special email promotions.

```

SELECT
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.email,
    g.phone,
    g.nationality
FROM Guest g
WHERE g.email REGEXP '@gmail\\.com$'
ORDER BY g.last_name;

```

Query 5: [3+ TABLE JOIN] List all confirmed reservations with guest contact details

- This query shows active bookings together with guest phone numbers and emails.
- Receptionists use this to prepare welcome messages and contact guests before arrival.

```

SELECT
    r.reservation_id,
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.phone,
    g.email,
    ro.room_number,
    rt.type_name AS room_type,
    r.check_in_date,
    r.check_out_date,
    r.total_amount,
    r.status
FROM Reservation r
JOIN Guest g ON r.guest_id = g.guest_id
JOIN Room ro ON r.room_id = ro.room_id
JOIN Room_Type rt ON ro.room_type_id = rt.room_type_id
WHERE r.status IN ('Confirmed', 'Checked-In')
ORDER BY r.check_in_date ASC;

```

Query 6: [3+ TABLE COMPLICATED JOIN] Full guest spending report including extra services

- how much each guest has spent in total, helping the hotel identify their top spenders.
- Service charges are pre-aggregated per reservation to ensure accurate totals.

```

SELECT
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.nationality,
    COUNT(DISTINCT r.reservation_id) AS total_stays,
    SUM(r.total_amount) AS room_charges,
    COALESCE(SUM(svc.svc_total), 0) AS service_charges,
    SUM(r.total_amount) + COALESCE(SUM(svc.svc_total), 0) AS grand_total
FROM Guest g
JOIN Reservation r ON g.guest_id = r.guest_id
LEFT JOIN (
    SELECT
        rs.reservation_id,
        SUM(s.price * rs.quantity) AS svc_total
    FROM Reservation_Service rs
    JOIN Service s ON rs.service_id = s.service_id
    GROUP BY rs.reservation_id
) svc ON r.reservation_id = svc.reservation_id
WHERE r.status != 'Cancelled'
GROUP BY g.guest_id, g.first_name, g.last_name, g.nationality
ORDER BY grand_total DESC;

```

Query 7: [NOT EXISTS + SUBQUERY] Find rooms that have NEVER been booked

- This query finds all rooms that do not appear in any reservation record.
- It helps the hotel manager spot unused rooms that may need promotion or inspection.

```

SELECT
    ro.room_number,
    rt.type_name AS room_type,
    ro.floor_number,
    ro.status,
    rt.base_price AS price_per_night
FROM Room ro
JOIN Room_Type rt ON ro.room_type_id = rt.room_type_id
WHERE NOT EXISTS (
    SELECT 1
    FROM Reservation r
    WHERE r.room_id = ro.room_id
        AND r.status != 'Cancelled'
)
ORDER BY ro.room_number;

```

Query 8: [GROUP BY + HAVING] Find guests who have made more than 1 reservation

- This query lists guests who have come back to the hotel more than once.
- It helps the hotel identify loyal returning guests who deserve special treatment or discounts.

```

SELECT
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.email,
    g.phone,
    g.nationality,
    COUNT(r.reservation_id) AS total_reservations,
    SUM(r.total_amount) AS total_spent
FROM Guest g
JOIN Reservation r ON g.guest_id = r.guest_id
WHERE r.status != 'Cancelled'
GROUP BY g.guest_id, g.first_name, g.last_name, g.email, g.phone, g.nationality
HAVING COUNT(r.reservation_id) > 1
ORDER BY total_reservations DESC;

```

Query 9: [GROUP BY + HAVING] Find room types that earned more than \$500 in total

- This query shows which room categories are bringing in the most revenue.
- Management uses this to focus marketing efforts on the most profitable room types.

```

SELECT
    rt.type_name          AS room_type,
    rt.base_price         AS price_per_night,
    COUNT(r.reservation_id) AS total_bookings,
    SUM(r.total_amount)    AS total_revenue
FROM Room_Type rt
JOIN Room      ro ON rt.room_type_id = ro.room_type_id
JOIN Reservation r ON ro.room_id      = r.room_id
WHERE r.status != 'Cancelled'
GROUP BY rt.room_type_id, rt.type_name, rt.base_price
HAVING SUM(r.total_amount) > 500
ORDER BY total_revenue DESC;

```

Query 10: [REGEXP] Find guests whose phone number starts with '012'

- This query searches for all guests whose phone number begins with the prefix '012'.
- It is useful when the hotel needs to contact a specific group of guests by phone prefix.

```

SELECT
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    g.phone,
    g.email,
    g.nationality
FROM Guest g
WHERE g.phone REGEXP '^012'
ORDER BY g.last_name;

```

Chapter 5: Conclusion & Future Enhancements

5.1 Summary of the database system

This database was built to help a hotel manage everything in one place. It stores information about room types and rooms, guests, staff, reservations, payments, and extra services like spa or room service. All these parts are connected to each other, so for example a reservation links a guest, a room, and a staff member together automatically.

It also supports useful queries like checking which rooms are available, how much revenue was made each month, which guests are returning customers, and how much each guest spent in total including extra services.

Overall it is a simple but solid system that covers everything a hotel needs for its day to day operations.

5.2 Suggested future improvements or extensions

- **Login and user roles:** Create a proper login system. Each staff member should only see and use the parts of the system that match their job.
- **Maintenance history:** When a room has a problem, don't just mark it as "Maintenance." Record what the problem was, when it was fixed, and which staff member fixed it.
- **Guest reviews:** Allow guests to leave a rating and a comment after they check out. This helps management understand guest satisfaction and improve service.
- **Better cancellation tracking:** When a booking is cancelled, record the reason and whether a refund was given, not just change the booking status.
- **Dynamic pricing:** Room prices should change depending on the season or demand, instead of having one fixed price all year.
- **Automatic notifications:** Send automatic messages to guests, such as booking confirmation, check in reminders, and a thank you message after their stay, by email or SMS.